

O projeto valerá de 0 a 10 e será levada em consideração a organização dos códigos de cada solução. Escreva códigos legíveis e com comentários sobre cada decisão importante feita nos algoritmos. As questões deverão ser implementadas em pthreads e utilizando o sistema operacional Linux. Ademais, caso uma questão necessite de arquivos, a equipe deverá disponibilizar arquivos exemplos de entrada. O não cumprimento das regras acarretará em perdas de pontos na nota final.

Atenção: Usar somente pthreads e Linux.

1. Considere um vetor grande de números inteiros (ex: 10.000 elementos). Implemente um programa cujo a soma é dividida entre N threads, e cada *thread* é responsável em somar um pedaço do vetor. A thread principal deve então somar os resultados parciais.

2. Você deve implementar uma tabela de chamada de pacientes em um consultório médico. A tabela deve ser apresentada no início e apenas as linhas necessárias devem ser atualizadas (pesquise sobre sequências de escape ANSI). Cada linha é independente, ou seja, linhas diferentes podem ser atualizadas ao mesmo tempo. Além disso, **deve haver exclusão mútua por linha**, logo, uma thread só pode modificar uma linha se ela estiver livre. Uma linha deve ficar alguns segundos bloqueada antes de ser modificada novamente para que a mudança seja visível. As informações sobre novos pacientes devem ser lidas de arquivos de texto externos (você deve criá-los). Neles, devem ser informados os nomes dos pacientes e o número do consultório. Haverá N arquivos com L linhas cada.

Exemplo de tabela inicial:

Paciente Gabro	Consultório 1
Paciente Feldspato	Consultório 2
Paciente Cherte	Consultório 3
Paciente Riebeck	Consultório 4
Paciente Esker	Consultório 5
Paciente Gneiss	Consultório 6
Paciente Corneana	Consultório 7

Exemplo de arquivo de texto:

Hal 6
Mica 2
Galena 5

Tabela após a leitura do arquivo:

Paciente Gabro	Consultório 1
Paciente Mica	Consultório 2
Paciente Cherte	Consultório 3
Paciente Riebeck	Consultório 4
Paciente Galena	Consultório 5
Paciente Hal	Consultório 6
Paciente Corneana	Consultório 7

3. O sistema operacional Linux comumente precisa lidar com várias operações de leituras e escritas concorrentes dentro do *kernel*. Neste contexto, podemos classificar as threads como leitoras e escritoras. Assuma que enquanto o *kernel* está sendo atualizado devido a uma operação de escrita (uma escritora), as threads leitoras precisam ser proibidas em realizar leitura no banco de dados. Isso é necessário para evitar que uma leitora interrompa uma modificação em progresso ou leia um dado inconsistente ou inválido.

Você deverá implementar um programa usando pthreads, considerando **N** threads leitoras e **M** threads escritoras. A região crítica deverá ser um array, e threads escritoras deverão continuamente (em um laço infinito) escrever no array em qualquer posição. Similarmente, as threads leitoras deverão ler dados (de forma contínua) de qualquer posição do array. As seguintes restrições deverão ser implementadas:

1. As threads leitoras podem simultaneamente acessar a região crítica (array). Ou seja, uma thread leitora não bloqueia outra thread leitora;
 2. Threads escritoras precisam ter acesso exclusivo à região crítica. Ou seja, a manipulação deve ser feita usando exclusão mútua. Ao entrar na região crítica, uma thread escritora deverá bloquear todas as outras threads escritoras e threads leitoras que desejarem acessar o recurso compartilhado.
4. O método de Jacobi é uma técnica representativa para solucionar sistemas de equações lineares (SEL). Um sistema de equações lineares possui o seguinte formato : $Ax = b$, no qual

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

Ex:

$$2x_1 + x_2 = 11$$

$$5x_1 + 7x_2 = 13$$

$$A = \begin{bmatrix} 2 & 1 \\ 5 & 7 \end{bmatrix}, \quad b = \begin{bmatrix} 11 \\ 13 \end{bmatrix} \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

O método de Jacobi assume uma solução inicial para as incógnitas (x_i) e o resultado é refinado durante P iterações, usando o algoritmo abaixo:

```
while(k < P)
begin
```

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right), \quad i = 1, 2, \dots, n.$$

```
    k = k + 1;
```

```
end
```

Por exemplo, assumindo o SEL apresentado anteriormente, P=10, e $x_1^{(0)}=1$ e $x_2^{(0)}=1$:

```
while(k < 10)
```

```
begin
```

$$x_1^{(k+1)} = 1/2 * (11 - x_2^{(k)})$$

$$x_2^{(k+1)} = 1/7 * (13 - 5x_1^{(k)})$$

```
    k = k+1;
```

```
end
```

Exemplo de execução

k=0

$$x_1^{(1)} = 1/2 * (11 - x_2^{(0)}) = 1/2 * (11-1) = 5$$

$$x_2^{(1)} = 1/7 * (13 - 5x_1^{(0)}) = 1/7 * (13-5 * 1) = 1.1428$$

k=1

$$x_1^{(2)} = 1/2 * (11 - 1.1428)$$

$$x_2^{(2)} = 1/7 * (13 - 5 * 5)$$

...

Nesta questão, o objetivo é quebrar a execução seqüencial em threads, na qual o valor de cada incógnita x_i pode ser calculado de forma concorrente em relação às demais incógnitas (Ex: $x_1^{(k+1)}$ pode ser calculada ao mesmo tempo que $x_2^{(k+1)}$). A quantidade de threads a serem criadas vai depender de um parâmetro N passado pelo usuário durante a execução do programa, e N deverá ser equivalente à quantidade de processadores (ou núcleos) que a máquina possuir. No início do programa, as N threads deverão ser criadas, I incógnitas igualmente associadas para thread, e nenhuma thread poderá ser instanciada durante a execução do algoritmo. Dependendo do número N de threads, alguma thread poderá ficar com menos incógnitas associadas à ela.

Para facilitar a construção do programa e a entrada de dados, as matrizes não precisam ser lidas do teclado, ou seja, podem ser inicializadas diretamente dentro do programa (ex: inicialização estática de vetores). Ademais, **os valores iniciais de $x_i^{(0)}$ deverão ser iguais a 1**, e adote mecanismo (ex: *barriers*) para sincronizar as threads depois de cada iteração.

Faça a experimentação executando o programa em uma máquina com 4 processadores/núcleos, demonstrando a melhoria da execução do programa com 1, 2 e 4 threads.

ATENÇÃO: apesar de $x_1^{(k+1)}$ pode ser calculada ao mesmo tempo que $x_2^{(k+1)}$, $x_i^{(k+2)}$ só poderão ser calculadas quando todas incógnitas $x_i^{(k+1)}$ forem calculadas. Barriers são uma excelente ferramenta para essa questão.

5. Uma versão brasileira do Linux pretende implementar um novo algoritmo de escalonamento para threads em CPUs *multicore* (múltiplos núcleos). Para isso, deve seguir os requisitos abaixo:

- Uma constante N que representa a quantidade de núcleos do sistema computacional. Consequentemente, N representará a quantidade máximas de threads em execução;
- **lista_pronto** que representará uma fila das execuções pendentes das threads;
- Uma thread **escalonador**. Esta deverá pegar as threads da **lista_pronto**, e gerenciar a execução nos N núcleos. Assuma que o código das threads são representadas por uma função qualquer que termina (ex: não tem laço infinito). Se não houver thread para ser executada na **lista_pronto**, a thread **escalonador** dorme. Pelo menos uma thread na **lista_pronto**, faz com que o **escalonador** acorde e coloque a nova thread pra executar. Se por um acaso N threads estejam executando e existem threads na **lista_pronto**, somente quando uma thread concluir a execução, uma nova thread será executada.
- **Atenção:** a thread escalonador é interna do sistema operacional e escondida do usuário.

A implementação não poderá ter espera ocupada. A estrutura e funcionamento ficarão a critério da equipe, desde que siga os requisitos acima. Por exemplo, pode-se criar uma função **agendar** para adicionar uma *thread* a **lista_pronto** e acordar o escalonador quando necessário.

Você deverá utilizar variáveis de condição para evitar a espera ocupada. Lembre-se que essas variáveis precisam ser utilizadas em conjunto com mutexes. Mutexes deverão ser utilizados de forma refinada, no sentido que um recurso não deverá travar outro recurso independente.